

PROJE TANITIMI

Ders: Derleyici Tasarımı / Compiler Design

Proje Adı: MiniLL(1): Küçük Bir Dil için Lexer ve LL(1) Parser Tasarımı

1. Projenin Amacı

Bu projenin amacı, basit bir programlama dili üzerinden derleyici tasarımındaki temel aşamaları uygulamalı olarak öğrenmektir:

- Kaynak programı karakterlerden **token dizisine** çeviren **lexer (sözcüksel çözümleyici)** tasarlamak,
- Verilen dil tanımı için **LL(1) biçiminde bir gramer** ile çalışmak,
- Bu gramer için **FIRST** ve **FOLLOW** kümelerini hesaplamak,
- Buna göre **LL(1) parse tablosu** oluşturmak,
- Tabloya dayalı stack tabanlı veya recursive descent yaklaşımıyla **LL(1) parser** geliştirmek,
- **Başarılı parse** durumunda, ekrana hem
 - “Program başarıyla parse edildi.” Mesajını

hem de ilgili program için **leftmost derivation** (soldan türetim) adımlarını yazdırmaktır.

2. MiniLang: Tanımlanan Küçük Dil

Bu projede "MiniLang" adı verilen küçük bir dil kullanılacaktır. MiniLang, aşağıdaki özelliklere sahip bir dildir:

- Sadece int türünde **değişken tanımlama**
- **Atama ifadeleri**
- **Aritmetik ifadeler:** +, -, *, /
- **Ekrana yazdırma komutu:** print

Örnek MiniLang Programı

```
int x;  
int y;  
x = 3 + 4 * 2;  
y = (x - 5) / 2;  
print(x);  
print(y);
```

Bu örnek, dilin tipik kullanımını ve desteklenen yapıları göstermektedir.

Yapılacak İşlemler

Lexical (Tokenlar)

Amaç: Kaynak kodu karakter dizisi olarak okumak ve bunu anlamlı bir **token** dizisine dönüştürmek.

Lexer bu aşamada çalışır.

Projede token türlerinin belirlenmesi beklenmektedir.

- **Anahtar kelimeler**
- **Semboller**
- **Diğerleri**
- **Yok sayılacak karakterler**
- (İsteğe bağlı) // ile başlayan tek satırlık yorumlar yok sayılabilir.

Lexer:

- Girdi dosyasını okuyup uygun token dizisini üretmelidir.
- Hatalı / tanımsız karakter gördüğünde anlamlı bir **hata mesajı** vermelidir (mümkünse satır numarasıyla).

Sözdizim (Gramer)

Amaç: Token dizisinin dilin kurallarına uyup uymadığını kontrol eden **parser** aşamasını tanımlamak.

MiniLang dilinin sözdizimi bağlamdan bağımsız gramer ile tanımlanmalıdır.

Projenin Adımları (Süreç)

1. Lexer Tasarımı ve Gerçeklenmesi

- Token türleri için düzenli ifade (regex) tanımları çıkarınız.
- Seçtiğiniz programlama dilinde (C/C++/Java/Python vb.) lexer yazınız.
- Lexer:
 - Girdi dosyasını okuyarak **token dizisi** üretmelidir.
 - Hatalı karakter gördüğünde tokenen bağımsız bir **lexical error** mesajı vermelidir.

Bu aşamanın çıktısı: “token akışı”dır. Örneğin:

INT ID SEMICOLON ID ASSIGN NUMBER PLUS NUMBER SEMICOLON ... EOF

2. FIRST ve FOLLOW Kümelerinin Hesaplanması

- Gramerdeki tüm nonterminaler için:
 - **FIRST kümelerini** hesaplayınız. (Bir nonterminalden türetilen ilk terminalleri gösterir.)
 - **FOLLOW kümelerini** hesaplayınız. (Bir nonterminalin sağında gelebilecek terminalleri gösterir.)
- Sonuçları raporda **tablo halinde** sununuz.

Bu aşama, LL(1) olup olmadığını kontrol etmek ve parse tablosunu doğru kurmak için gereklidir.

3. LL(1) Parse Tablosunun Oluşturulması

- FIRST ve FOLLOW kümelerini kullanarak **LL(1) parse tablosu** oluşturunuz.
- Her hücrede hangi üretimin kullanılacağı açıkça belirtilmelidir.
- Tabloyu raporunuzda düzenli bir şekilde (satırlarda nonterminaler, sütunlarda terminaller) gösteriniz.

Bu aşamanın çıktısı: Parser’ın bakarak karar vereceği **LL(1) parse tablosu**dur.

4. LL(1) Parser Gerçeklenmesi

Amaç: Token dizisini okuyup, verilen gramera göre bir parse süreci yürütmek.

İki yaklaşımdan birini seçebilirsiniz:

1. **Tabloya dayalı stack tabanlı LL(1) parser**
 - Başlangıçta stack: [Program, EOF]
 - Girdi token akışını okuyarak:
 - Üstte nonterminal varsa parse tablosuna bakıp uygun üretimi stack’e açınız.
 - Üstte terminal varsa ve mevcut token ile eşleşiyorsa her ikisini tüketiniz.
 - Eşleşme yoksa syntax error üretiniz.
2. **Recursive descent parser**
 - Her nonterminal için bir fonksiyon tanımlayınız (ör: parseProgram, parseStmtList, parseStmt, ...).
 - FIRST/FOLLOW kümelerine uygun şekilde dallanmaları ve ϵ -üretimlerini yönetiniz.
 - Hata durumunda anlamlı syntax error mesajı üretiniz.

5. Program Çıktısı ve Leftmost Derivation

Parser’ın **geçerli** bir programı tamamen başarıyla okuması durumunda:

1. Ekranı **başarı mesajı** yazdırılmalıdır:

Program başarıyla parse edildi.

2. Ardından, aynı program için **leftmost derivation (soldan türetim)** adımlarını yazdırılmalıdır.

Leftmost derivation:

Her adımda, üretim uygulanırken **en soldaki nonterminal** genişletilir.

Parser'ın **hatalı** bir programla karşılaşması durumunda:

- Başarı mesajı ve leftmost derivation **yazılmamalı**, bunun yerine anlamlı bir **syntax error** mesajı üretilmelidir.

6. Teslim Edilecekler

1. Kaynak Kodlar

- Lexer ve parser'ın tüm kaynak dosyaları
- Programı nasıl derleyip çalıştıracağınızı açıklayan kısa bir README dosyası

2. Proje Raporu (PDF)

- MiniLang dilinin kısa tanımı
- Kullanılan token listesi ve kısa açıklamaları
- FIRST ve FOLLOW kümeleri (tablolar)
- LL(1) parse tablosu
- En az bir **geçerli** örnek program için:
 - Program çıktısı (başarı mesajı + leftmost derivation)
- En az bir **hatalı** program için:
 - Program çıktısı (syntax error mesajı)

3. Örnek Girdi Dosyaları

- Birkaç geçerli MiniLang programı
- Birkaç hatalı MiniLang programı

Değerlendirme Ölçütleri

- **Doğruluk ve Tamlık**
 - Lexer ve parser'ın verilen dil tanımını doğru işlemesi
 - FIRST/FOLLOW ve LL(1) parse tablosunun tutarlılığı
- **Çıktı Formatı**
 - Başarılı parse sonrası:
 - "Program başarıyla parse edildi. " mesajının üretilmesi
 - Leftmost derivation adımlarının açık ve anlaşılır bir biçimde gösterilmesi
- **Rapor Kalitesi**
 - Açıklamaların netliği, tabloların düzeni, örneklerin uygunluğu

MiniLL(1) Projesi Teslim Duyurusu

- **Dijital teslim son tarihi:** 7 Aralık 2025, saat 23:59
- **Teslim yeri:** Google Classroom üzerinden ilgili ödev alanı

Bu tarihi takip eden **hafta ders saatinde** projeler öğrenciler tarafından **yüzyüze** teslim edilecektir. Ders saatinde **yüzyüze teslim yapmayan** öğrencilerin proje notu "**GİRMEDİ**" olarak işlenecektir.